

WAF^T

TYPST TEMPLATE LIBRARY

Exhaustive Menu of ALL Visualization Options

8+

Packages

50+

Examples

10

Categories

100%

Live

Every example in this document is **LIVE** — generated by the actual Typst package, not screenshots or simulations.

Packages Demonstrated: gentle-clues • zero • lilaq • fletcher • tablex • diagraph • pintorita • algorithmic

Table of Contents

1	Part 1: Callouts & Admonitions	3
1.1	1.1 Standard Callout Types	3
1.2	1.2 Callouts with Content	3
2	Part 2: Scientific Number Formatting	4
2.1	2.1 Basic Number Formatting	4
2.2	2.2 Uncertainties	4
2.3	2.3 Units (zi module)	4
2.4	2.4 In-Context Usage	4
3	Part 3: Scientific Plotting	5
3.1	3.1 Line Plots	5
3.2	3.2 Scatter Plots	6
3.3	3.3 Bar Charts	6
3.4	3.4 Multiple Series	6
4	Part 4: Node-Edge Diagrams (fletcher)	7
4.1	4.1 Simple Flowcharts	7
4.2	4.2 Vertical Flow	7
4.3	4.3 Branching Decision Tree	7
4.4	4.4 Multi-Agent Architecture	8
4.5	4.5 Data Pipeline	8
4.6	4.6 WAFT Evolution Cycle	8
5	Part 5: Graphviz Diagrams (diagraph)	9
5.1	5.1 Simple Directed Graph	9
5.2	5.2 Labeled Graph	9
5.3	5.3 Styled Graph	9
5.4	5.4 Cluster Subgraphs	10
5.5	5.5 Complex System Architecture	11
6	Part 6: Pintora Diagrams (Mermaid-like)	12
6.1	6.1 Sequence Diagram	12
6.2	6.2 Entity Relationship Diagram	12
6.3	6.3 Component Diagram	13
6.4	6.4 Activity Diagram	14
6.5	6.5 Mind Map	15
7	Part 7: Advanced Tables (tablex)	16
7.1	7.1 Styled Headers	16
7.2	7.2 Row and Column Spans	16
7.3	7.3 Progress Bars in Tables	16
7.4	7.4 Alternating Row Colors	17
7.5	7.5 Comparison Table	17
8	Part 8: Algorithm Pseudocode	18
8.1	8.1 Basic Algorithm	18
8.2	8.2 Conditional Logic	18
9	Part 9: Native Typst Features	19
9.1	9.1 Math Equations	19
9.2	9.2 Code Blocks	19
9.3	9.3 Lists	19
10	Part 10: Combined Examples	20
10.1	10.1 Complete Status Report	20
10.2	10.2 System Architecture + Flow	21
11	Appendix: Import Cheatsheet	22
11.1	11.1 A.1 All Imports	22
11.2	11.2 A.2 Quick Reference	22

1 Part 1: Callouts & Admonitions

Package: gentle-clues:1.2.0

1.1 1.1 Standard Callout Types

Info

Info — General information for readers.

Success

Success — Confirmations and positive outcomes.

Warning

Warning — Cautions about potential issues.

Error

Error — Critical errors or failures.

Tip

Tip — Helpful suggestions and best practices.

Memorize

Memo — Notes and reminders.

Question

Question — Uncertainties or open questions.

Example

Example — Demonstrations and illustrations.

1.2 1.2 Callouts with Content

Info

Multi-line Info Box

You can include:

- Bullet points
- **Bold text**
- *Italic text*
- Code snippets

Tip

Code Example Inside Tip

```
def hello():  
    print("Hello WAFT!")
```

2 Part 2: Scientific Number Formatting

Package: zero:0.5.0

2.1 2.1 Basic Number Formatting

Input	Code	Output
Large	<code>#num[1234567]</code>	1 234 567
Decimal	<code>#num[3.14159]</code>	3.141 59
Scientific	<code>#num[6.022e23]</code>	6.022×10^{23}
Negative exp	<code>#num[1.38e-23]</code>	1.38×10^{-23}
Speed of light	<code>#num[299792458]</code>	299 792 458

Figure 1: Basic Number Formatting

2.2 2.2 Uncertainties

Code	Output
<code>#num("9.81+-0.02")</code>	9.81 ± 0.02
<code>#num("1.602+-0.001e-19")</code>	$(1.602 \pm 0.001) \times 10^{-19}$
<code>#num("2.998(1)e8")</code>	$(2.998 \pm 0.001) \times 10^8$
<code>#num("42.0+-0.5")</code>	42.0 ± 0.5

Figure 2: Numbers with Uncertainties

2.3 2.3 Units (zi module)

Quantity	Code	Output
Time	<code>#zi.s[9.58]</code>	9.58 s
Length	<code>#zi.m[100]</code>	100 m
Mass	<code>#zi.kg[75.5]</code>	75.5 kg
Frequency	<code>#zi.Hz[440]</code>	440 Hz

Figure 3: Physical Units

2.4 2.4 In-Context Usage

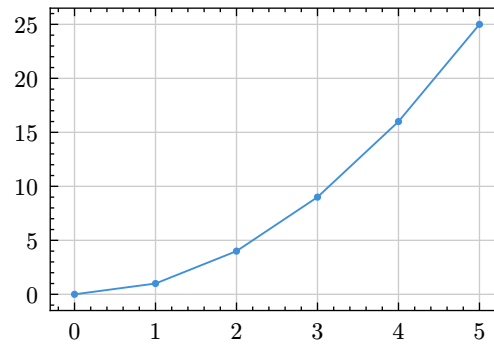
The speed of light is 299 792 458 m/s. Planck's constant is 6.626×10^{-34} J · s. Usain Bolt's 100m world record: 9.58 s.

3 Part 3: Scientific Plotting

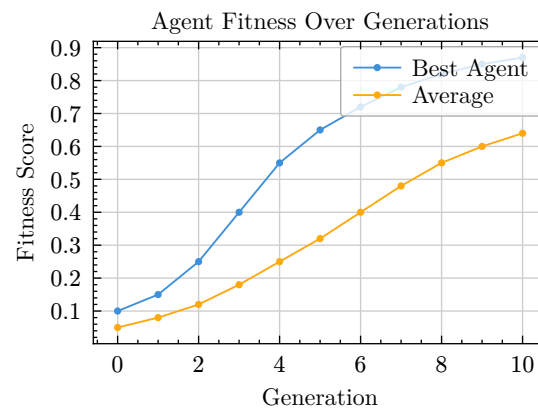
Package: lilaq:0.5.0

3.1 3.1 Line Plots

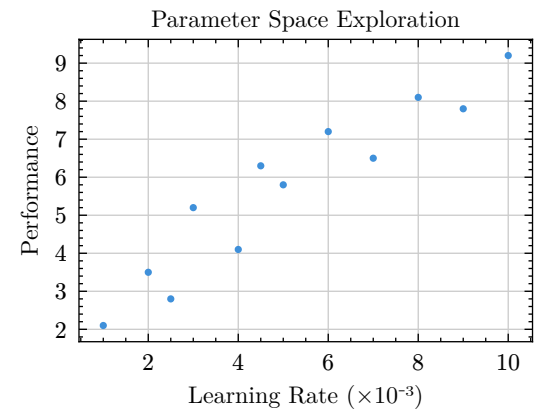
3.1.1 Basic Line Plot



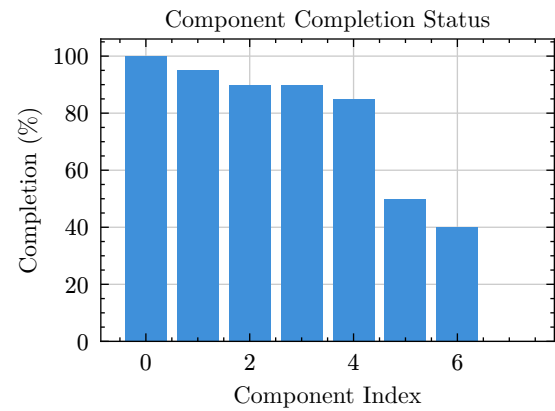
3.1.2 Line Plot with Labels



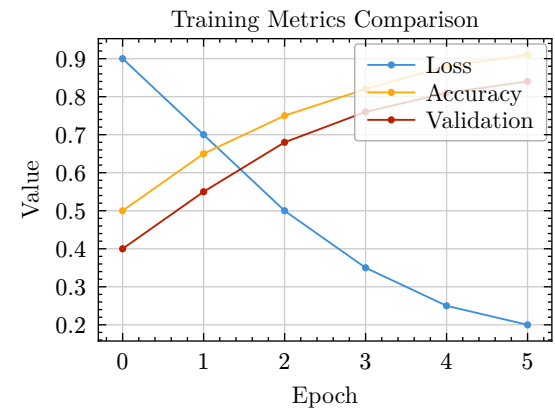
3.2 3.2 Scatter Plots



3.3 3.3 Bar Charts



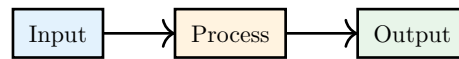
3.4 3.4 Multiple Series



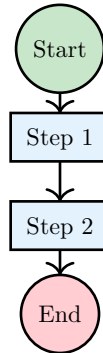
4 Part 4: Node-Edge Diagrams (fletcher)

Package: fletcher:0.5.8

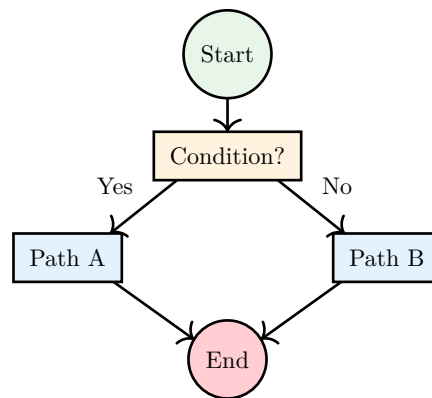
4.1 4.1 Simple Flowcharts



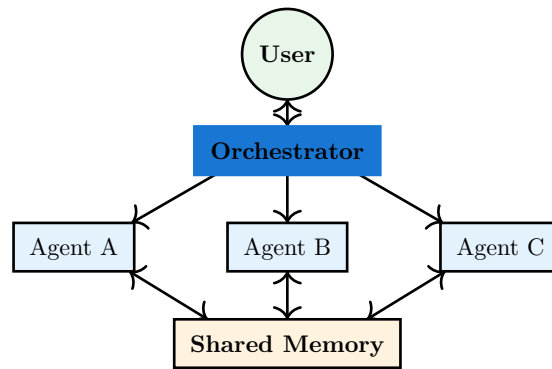
4.2 4.2 Vertical Flow



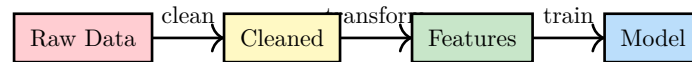
4.3 4.3 Branching Decision Tree



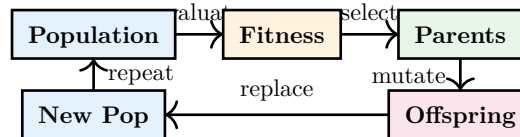
4.4 4.4 Multi-Agent Architecture



4.5 4.5 Data Pipeline



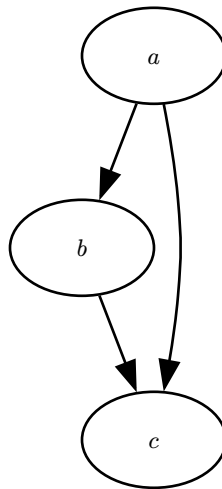
4.6 4.6 WAFT Evolution Cycle



5 Part 5: Graphviz Diagrams (diagraph)

Package: `diagraph:0.3.6`

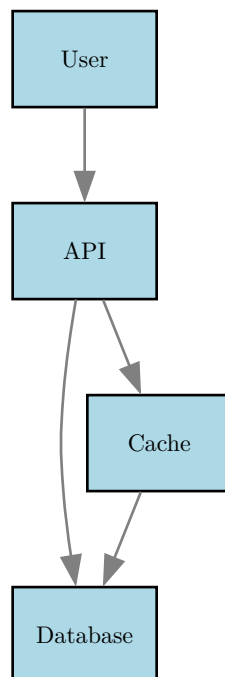
5.1 5.1 Simple Directed Graph



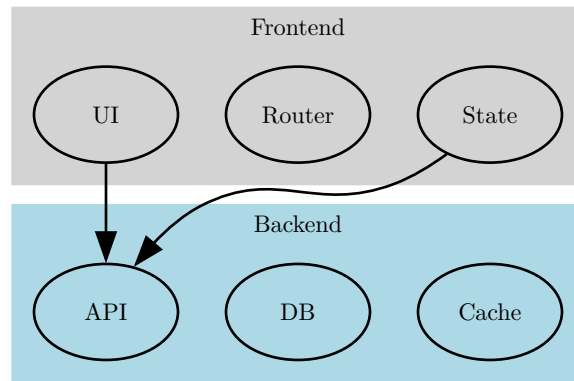
5.2 5.2 Labeled Graph



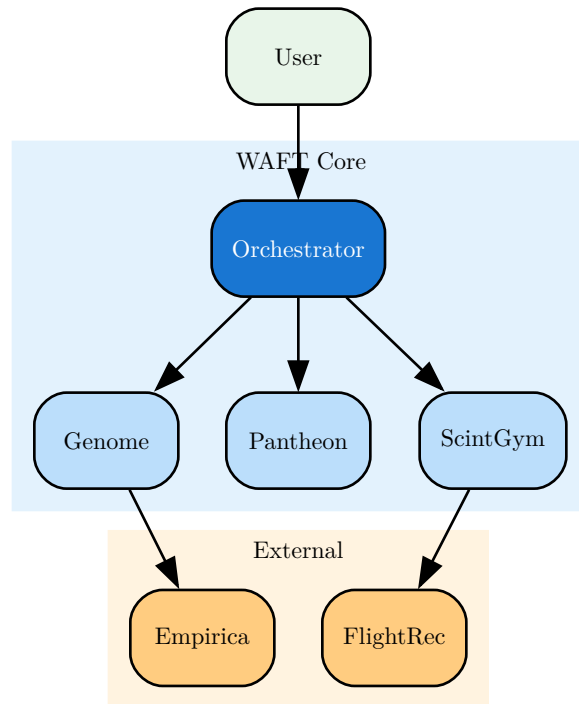
5.3 5.3 Styled Graph



5.4 5.4 Cluster Subgraphs



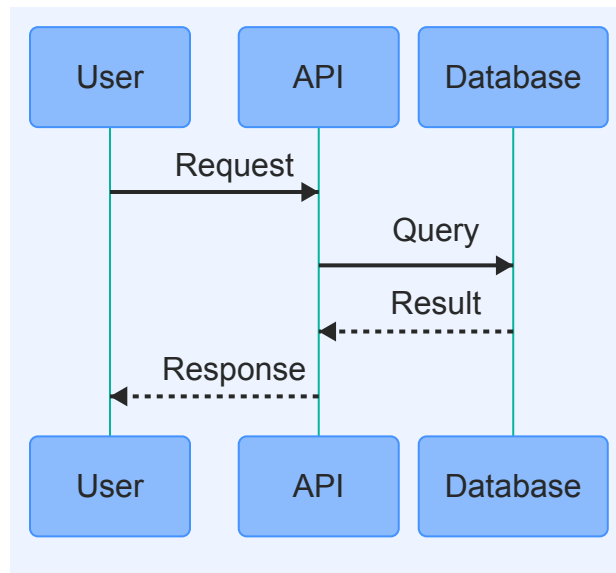
5.5 5.5 Complex System Architecture



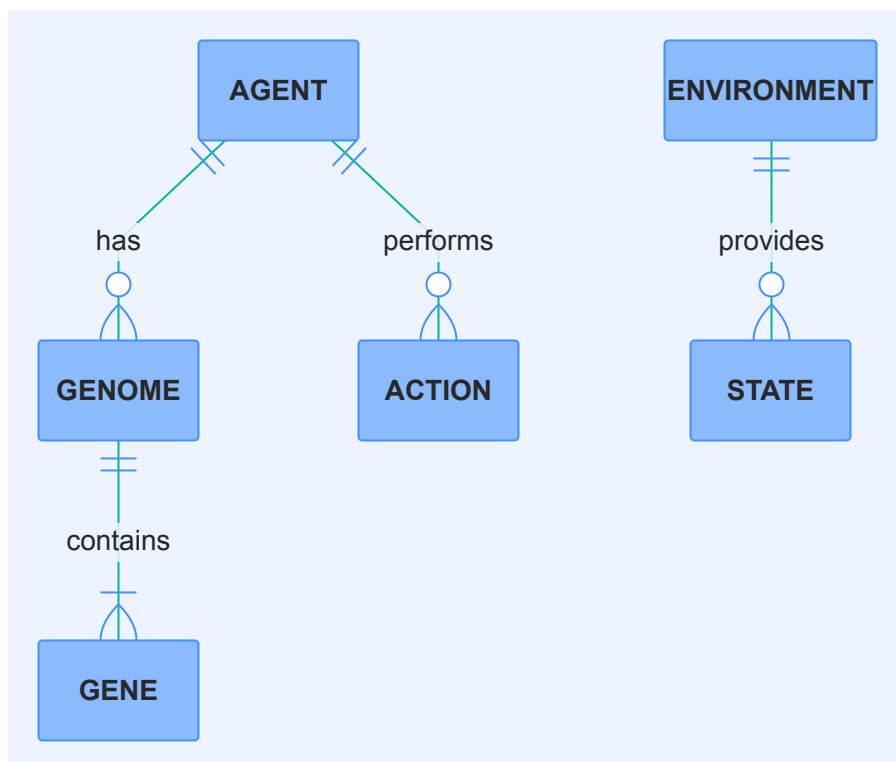
6 Part 6: Pintora Diagrams (Mermaid-like)

Package: pintorita:0.1.4

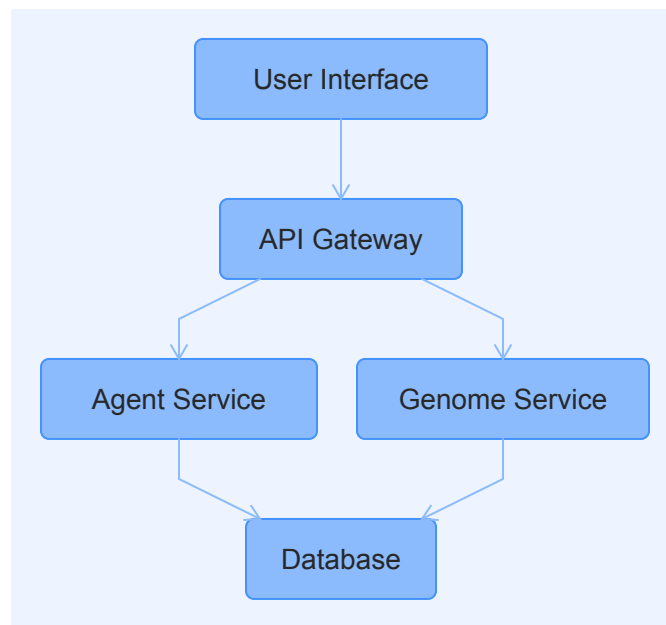
6.1 6.1 Sequence Diagram



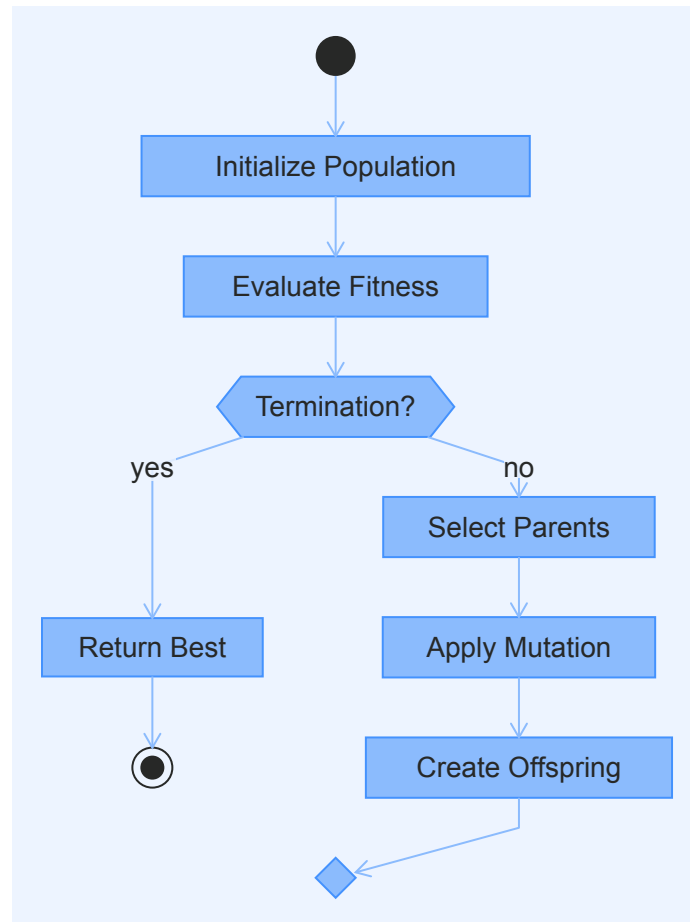
6.2 6.2 Entity Relationship Diagram



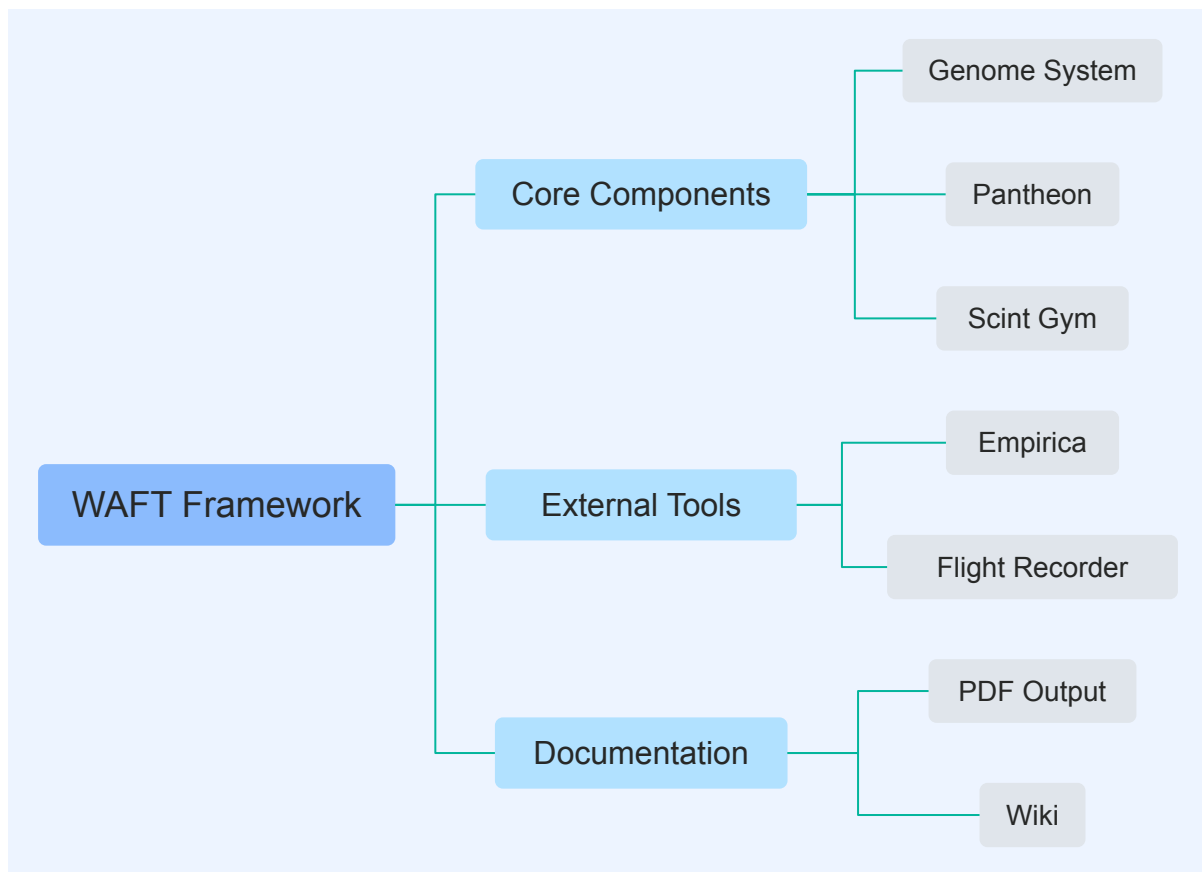
6.3 6.3 Component Diagram



6.4 6.4 Activity Diagram



6.5 6.5 Mind Map



7 Part 7: Advanced Tables (tablex)

Package: tablex:0.0.9

7.1 7.1 Styled Headers

ID	Component	Status	✓
1	Empirica Integration	100%	✓
2	Genome System	95%	✓
3	RPG Gym	90%	✓
4	Multi-Agent	50%	⚠
5	Evolution Cycle	0%	✗

Figure 4: Styled Header Table

7.2 7.2 Row and Column Spans

WAFT Framework Components			
Core	Genome	SHA-256	✓
	Pantheon	Beings	✓
Tools	Empirica	External	✓
	Flight Rec	Telemetry	✓
Output	PDF	Typst	✓
	Wiki	Markdown	✓

Figure 5: Table with Row/Column Spans

7.3 7.3 Progress Bars in Tables

Component	Progress	%
Empirica		100%
Genome		95%
RPG Gym		90%
Pantheon		90%
Flight Rec		85%
Multi-Agent		50%
Mutation		40%
Evo Cycle		0%

Figure 6: Progress Visualization in Tables

7.4 7.4 Alternating Row Colors

No.	Description	Value
1	Total lines added	2,760+
2	Files created	9
3	Wiki pages	4
4	Git commits	2
5	Packages used	8

Figure 7: Alternating Row Colors

7.5 7.5 Comparison Table

Feature	WAFT	Traditional	Diff
Epistemic Tracking	✓	✗	+100%
Genome Versioning	✓	✗	+100%
Multi-Agent	⚠	✗	+50%
Evolution Cycle	✗	✗	0%

Figure 8: Feature Comparison

8 Part 8: Algorithm Pseudocode

Package: algorithmic:1.0.7

8.1 8.1 Basic Algorithm

Algorithm 1: WAFT Evolution Loop

```

1: 1: function EVOLVE(population, generations)
2:   for  $i \leq$  generations do
3:     fitness  $\leftarrow$  Evaluate(population)
4:     parents  $\leftarrow$  Select(population, fitness)
5:     offspring  $\leftarrow$  Mutate(parents)
6:     population  $\leftarrow$  Replace(population, offspring)
7:   end
8:   return BestIndividual(population)
9: end

```

8.2 8.2 Conditional Logic

Algorithm 2: Fitness Evaluation

```

1: 1: function EVALUATEFITNESS(agent, environment)
2:   score  $\leftarrow$  0
3:   for task  $\in$  environment do
4:     result  $\leftarrow$  agent.Execute(task)
5:     if result = success then
6:       score  $\leftarrow$  score + 1
7:     end
8:     else
9:       score  $\leftarrow$  score - 0.5
10:    end
11:  end
12:  return score
13: end

```

9 Part 9: Native Typst Features

9.1 9.1 Math Equations

9.1.1 Inline Math

The quadratic formula is $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

9.1.2 Display Math

$$E = mc^2$$

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

$$\sum_{n=1}^\infty \frac{1}{n^2} = \frac{\pi^2}{6}$$

9.1.3 Matrix

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

9.2 9.2 Code Blocks

```
def evolve(population, generations):
    for _ in range(generations):
        fitness = evaluate(population)
        parents = select(population, fitness)
        offspring = mutate(parents)
        population = replace(population, offspring)
    return best(population)

fn main() {
    let waft = WaftFramework::new();
    waft.evolve(100); // 100 generations
    println!("Best fitness: {}", waft.best_fitness());
}
```

9.3 9.3 Lists

9.3.1 Bullet List

- First item
- Second item
 - Nested item
 - Another nested
- Third item

9.3.2 Numbered List

1. Step one
2. Step two
3. Step three

9.3.3 Definition List

WAFt Workbench for Agent Fitness Testing

Genome Unique identifier and lineage tracker

Pantheon Collection of entity archetypes

10 Part 10: Combined Examples

10.1 10.1 Complete Status Report

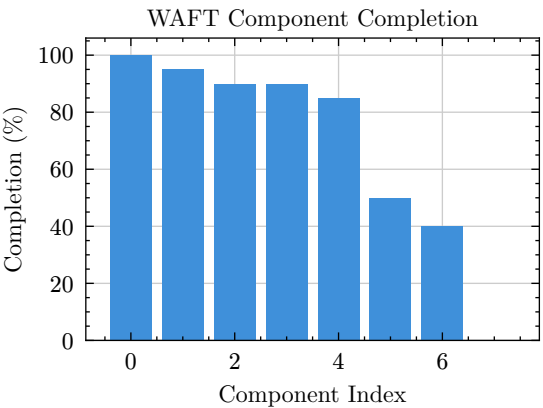
i

Info

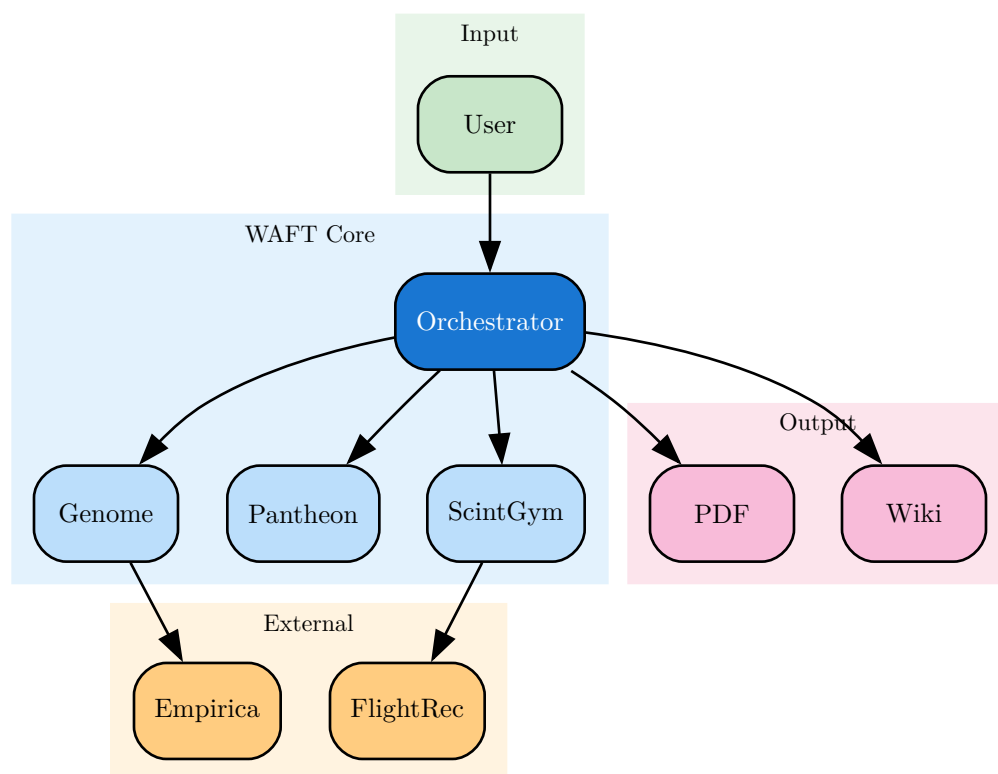
WAFt Framework Status Report — January 24, 2026

ID	Component	%	Progress	Status
1	Empirica	100		✓
2	Genome	95		✓
3	RPG Gym	90		✓
4	Pantheon	90		✓
5	Flight Rec	85		✓
6	Multi-Agent	50		⚠
7	Mutation	40		⚠
8	Evo Cycle	0		✗

Figure 9: Complete WAFt Status with Numbers, Progress Bars, and Status Icons



10.2 10.2 System Architecture + Flow



✓ Success

All diagrams, tables, charts, and visualizations above are generated by **real Typst packages**, not simulated.

11 Appendix: Import Cheatsheet

11.1 A.1 All Imports

```
// Callouts
#import "@preview/gentle-clues:1.2.0": *
```

```
// Scientific numbers and units
#import "@preview/zero:0.5.0": num, zi
```

```
// Scientific plotting
#import "@preview/lilaq:0.5.0" as lq
```

```
// Node-edge diagrams
#import "@preview/fletcher:0.5.8" as fletcher: diagram, node, edge
```

```
// Advanced tables
#import "@preview/tablex:0.0.9": tablex, cellx, rowspanx, colspanx
```

```
// Graphviz diagrams
#import "@preview/diagraph:0.3.6": render as graphviz-render
```

```
// Pintora (Mermaid-like) diagrams
#import "@preview/pintorita:0.1.4"
```

```
// Pseudocode
#import "@preview/algorithmic:1.0.7": algorithm, algorithm-figure
```

11.2 A.2 Quick Reference

Package	Use For	Key Function
gentle-clues	Callouts	#info[], #warning[]
zero	Numbers/Units	#num[], #zi.s[]
lilaq	Plots	#lq.diagram(), #lq.plot()
fletcher	Flowcharts	#diagram(), #node(), #edge()
tablex	Tables	#tablex(), #cellx()
diagraph	Graphviz	#graphviz-render()
pintorita	Mermaid	#pintorita.render()
algorithmic	Pseudocode	#algorithm-figure()

Figure 10: Quick Reference Guide

WAFT Typst Template Library v1.0

Every visualization is REAL. Copy and use.

Generated: 2026-01-24 | Source: Typst Universe